21.1 扩展架构设计

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~700k tokens，输出 ~65k tokens（本章合计）

前两章我们深入分析了 OpenClaw 内置的六大核心通道（WhatsApp、Telegram、Discord、Slack、Signal、iMessage）。但通讯平台数以百计，不可能全部内置。OpenClaw 通过**插件系统**（Plugin System）实现了通道的可扩展性——任何人都可以编写一个插件来接入新的通讯平台。本章将深入剖析这套扩展机制的设计与实现。

21.1.1 扩展与核心通道的区别

OpenClaw 中的通道分为两类：

维度	核心通道（Core Channels）	扩展通道（Extension Channels）
代码位置	`src/web/` 、 `src/telegram/` 等	`extensions/` 或 `~/.config/openclaw/extensions/`
加载方式	编译时静态导入	运行时通过 jiti 动态加载
配置位置	`channels.slack` 、 `channels.telegram` 等	`plugins.entries.<id>`
发布方式	随 OpenClaw 主包发布	npm 包或本地目录
监听方式	直接调用 `monitor*Provider()`	通过 Plugin Service 或 Hook 注册
示例	WhatsApp、Telegram、Discord、Slack	MS Teams、Matrix、Twitch、飞书

这个区分不是能力层面的——扩展通道可以实现与核心通道完全相同的功能。区分的本质是**部署策略**：核心通道是 OpenClaw 的"出厂配置"，而扩展通道是"按需安装"。

为什么需要扩展?

1. **减小主包体积**：不是每个用户都需要飞书或 Matrix
2. **独立版本管理**：扩展可以独立于主包更新
3. **社区贡献友好**：第三方可以不修改主仓库就发布新通道
4. **原生依赖隔离**：某些通道（如 Matrix）需要 native 模块（ `matrix-sdk-crypto-nodejs` ），放在扩展中可以避免安装失败影响主包

21.1.2 扩展加载机制（ `src/plugins/loader.ts` ）

加载流程概览

插件的加载由 `loadOpenClawPlugins()` 函数编排，这是整个插件系统的核心入口：

```typescript
// src/plugins/loader.ts（简化）
export function loadOpenClawPlugins(options: PluginLoadOptions = {}): PluginRegistry {
  // 1. 规范化插件配置
  const normalized = normalizePluginsConfig(cfg.plugins);

  // 2. 检查缓存（避免重复加载）
  const cacheKey = buildCacheKey({ workspaceDir, plugins: normalized });
  if (registryCache.has(cacheKey)) return registryCache.get(cacheKey);

  // 3. 发现所有候选插件
  const discovery = discoverOpenClawPlugins({ workspaceDir, extraPaths: normalized.load

  // 4. 加载插件清单（manifest）
  const manifestRegistry = loadPluginManifestRegistry({ config, candidates: discovery.c

  // 5. 初始化 jiti 运行时（支持 TypeScript 直接加载）
  const jiti = createJiti(import.meta.url, {
    interopDefault: true,
    extensions: [".ts", ".tsx", ".mts", ".cts", ".js", ".mjs", ".cjs", ".json"],
    alias: { "openclaw/plugin-sdk": pluginSdkAlias },
  });

  // 6. 逐个加载并注册插件
  for (const candidate of discovery.candidates) {
    // 6a. 检查启用状态
    const enableState = resolveEnableState(pluginId, candidate.origin, normalized);
    if (!enableState.enabled) continue;

    // 6b. 加载模块
    const mod = jiti(candidate.source);

    // 6c. 验证配置
    const validatedConfig = validatePluginConfig({ schema, value: entry?.config });

    // 6d. 调用 register() 注册
    const api = createApi(record, { config: cfg, pluginConfig: validatedConfig.value })
    register(api);
  }

  // 7. 设置全局 Hook Runner 并缓存
  initializeGlobalHookRunner(registry);
```

```
  return registry;
}
```

整个流程可以概括为四个阶段：

```
发现（Discovery）→ 解析（Manifest）→ 加载（jiti）→ 注册（register/activate）
```

插件发现（Discovery）

`discoverOpenClawPlugins()` 从四个位置搜索插件，优先级从高到低：

```
// src/plugins/discovery.ts
export function discoverOpenClawPlugins(params) {
  // 1. 配置中显式指定的路径（最高优先级）
  for (const extraPath of normalized.loadPaths) {
    discoverFromPath({ rawPath: trimmed, origin: "config" });
  }

  // 2. 工作区目录（项目级）
  const workspaceExtDirs = [path.join(workspaceRoot, ".openclaw", "extensions")];
  for (const dir of workspaceExtDirs) {
    discoverInDirectory({ dir, origin: "workspace" });
  }

  // 3. 全局配置目录（用户级）
  const globalDir = path.join(resolveConfigDir(), "extensions");
  discoverInDirectory({ dir: globalDir, origin: "global" });

  // 4. 内置插件目录（包级）
  const bundledDir = resolveBundledPluginsDir();
  discoverInDirectory({ dir: bundledDir, origin: "bundled" });
}
```

来源	路径	origin 标识	使用场景
配置路径	`plugins.loadPaths` 中指定	`"config"`	开发调试、自定义部署
工作区	`.openclaw/extensions/`	`"workspace"`	项目级插件
全局	`~/.config/openclaw/extensions/`	`"global"`	用户级插件
内置	OpenClaw 包中的 `extensions/`	`"bundled"`	官方扩展（MS Teams 等）

每个发现位置支持两种组织形式：

1. **单文件插件**：目录中直接放 `.ts` 或 `.js` 文件
2. **包形式插件**：子目录包含 `package.json`，其中 `openclaw.extensions` 字段声明入口文件

```
// 包形式插件的 package.json
{
  "name": "@openclaw/msteams",
  "openclaw": {
    "extensions": ["src/index.ts"]
  }
}
```

插件清单（Manifest）

每个插件必须包含一个 `openclaw.plugin.json` 清单文件：

```
{
  "id": "msteams",
  "name": "Microsoft Teams",
  "description": "MS Teams channel integration via Bot Framework",
  "version": "0.1.0",
  "kind": null,
  "channels": ["msteams"],
  "configSchema": {
    "type": "object",
    "properties": {
      "appId": { "type": "string" },
      "appPassword": { "type": "string" }
    },
    "required": ["appId", "appPassword"]
  }
}
```

清单的关键字段：

字段	必填	说明
`id`	✅	全局唯一标识符
`configSchema`	✅	JSON Schema，用于验证插件配置

字段	必填	说明
`kind`	❌	插件类型（目前仅 `"memory"` 用于记忆插件的互斥）
`channels`	❌	声明此插件提供的通道 ID
`providers`	❌	声明此插件提供的模型 Provider
`uiHints`	❌	配置字段的 UI 提示（用于控制面板）

jiti 动态加载

> **衍生解释**：jiti 是一个即时编译 TypeScript/ESM 的运行时工具。它允许 Node.js 直接 `require()` TypeScript 文件，无需预编译。这对插件系统特别有价值——用户可以直接用 TypeScript 编写插件，无需配置构建工具链。

OpenClaw 使用 jiti 加载插件模块：

```
const jiti = createJiti(import.meta.url, {
  interopDefault: true,
  extensions: [".ts", ".tsx", ".mts", ".cts", ".js", ".mjs", ".cjs", ".json"],
  alias: { "openclaw/plugin-sdk": pluginSdkAlias },
});

// 直接加载 TypeScript 文件
const mod = jiti(candidate.source) as OpenClawPluginModule;
```

`alias` 配置将 `openclaw/plugin-sdk` 映射到实际的 SDK 源文件路径，使得插件可以这样导入：

```
// 插件代码中
import type { OpenClawPluginApi, ChannelPlugin } from "openclaw/plugin-sdk";
```

启用/禁用控制

插件的启用状态由多层配置共同决定：

```
const enableState = resolveEnableState(pluginId, candidate.origin, normalized);
```

控制方式包括：

1. **显式启用**：`plugins.entries.<id>.enabled: true`

2. **显式禁用**： `plugins.entries.<id>.enabled: false`
3. **deny 列表**： `plugins.deny: ["msteams"]`
4. **allow 列表**： `plugins.allow: ["msteams"]` （如果设置，则只加载列出的插件）
5. **默认行为**： `bundled` 来源的插件默认启用，其他来源的插件默认启用

21.1.3 扩展目录结构分析（以 MS Teams 为例）

虽然我们无法在本书中展示所有扩展的源代码，但可以通过通用的扩展目录结构来理解扩展应该如何组织。一个典型的通道扩展目录结构如下：

```
extensions/msteams/
├── package.json                  # npm 包配置
│   └── "openclaw": { "extensions": ["src/index.ts"] }
├── openclaw.plugin.json          # 插件清单（必须）
├── src/
│   ├── index.ts                  # 入口：导出 register 函数
│   ├── config.ts                 # 配置解析与验证
│   ├── monitor.ts                # 消息监听（类似核心通道的 monitorProvider）
│   ├── send.ts                   # 消息发送
│   ├── format.ts                 # Markdown → 平台格式转换
│   ├── types.ts                  # 类型定义
│   └── adapters/
│       ├── config.ts             # ChannelConfigAdapter 实现
│       ├── outbound.ts           # ChannelOutboundAdapter 实现
│       ├── security.ts           # ChannelSecurityAdapter 实现
│       └── status.ts             # ChannelStatusAdapter 实现
└── tsconfig.json
```

入口文件的基本结构：

```typescript
// extensions/msteams/src/index.ts
import type { OpenClawPluginApi } from "openclaw/plugin-sdk";

export function register(api: OpenClawPluginApi) {
  const pluginConfig = api.pluginConfig;

  // 注册通道
  api.registerChannel({
    plugin: createMSTeamsPlugin(pluginConfig),
    dock: createMSTeamsDock(pluginConfig),
  });

  // 注册后台服务（消息监听循环）
  api.registerService({
    id: "msteams-monitor",
    start: async (ctx) => {
      await startMSTeamsMonitor(ctx, pluginConfig);
    },
    stop: async () => {
      await stopMSTeamsMonitor();
    },
  });

  // 注册 Hook（可选）
  api.on("message_sending", (event, ctx) => {
    // 在消息发送前做格式转换
  });
}
```

扩展与主程序的交互接口

扩展通过 `OpenClawPluginApi` 与 OpenClaw 主程序交互。这个 API 对象在 `register()` 被调用时注入，提供了注册各种能力的方法：

```
type OpenClawPluginApi = {
  id: string;                           // 插件 ID
  config: OpenClawConfig;               // 全局配置（只读）
  pluginConfig?: Record<string, unknown>; // 插件自身配置
  runtime: PluginRuntime;               // 运行时环境
  logger: PluginLogger;                 // 日志接口

  // 注册方法
  registerChannel(...);      // 注册通道适配器
  registerTool(...);         // 注册 Agent 工具
  registerHook(...);         // 注册事件钩子
  registerHttpHandler(...);  // 注册 HTTP 处理器
  registerHttpRoute(...);    // 注册 HTTP 路由
  registerGatewayMethod(...); // 注册 Gateway RPC 方法
  registerCli(...);          // 注册 CLI 子命令
  registerService(...);      // 注册后台服务
  registerProvider(...);     // 注册模型提供者
  registerCommand(...);      // 注册自定义命令

  // 生命周期钩子
  on(hookName, handler);     // 监听生命周期事件

  // 工具方法
  resolvePath(input);        // 解析相对路径
};
```

这个设计遵循了**控制反转**（IoC）原则——插件不直接调用 OpenClaw 的内部 API，而是通过注册回调的方式声明自己的能力，由 OpenClaw 的运行时在合适的时机调用。

加载到 Gateway 的流程

在 Gateway 服务器启动时， `loadGatewayPlugins()` 负责将插件系统接入 Gateway：

```typescript
// src/gateway/server-plugins.ts
export function loadGatewayPlugins(params) {
  const pluginRegistry = loadOpenClawPlugins({
    config: params.cfg,
    workspaceDir: params.workspaceDir,
    coreGatewayHandlers: params.coreGatewayHandlers,
  });

  // 合并插件注册的 Gateway 方法到主方法列表
  const pluginMethods = Object.keys(pluginRegistry.gatewayHandlers);
  const gatewayMethods = [...new Set([...params.baseMethods, ...pluginMethods])];

  // 输出诊断信息
  for (const diag of pluginRegistry.diagnostics) {
    if (diag.level === "error") params.log.error(`[plugins] ${diag.message}`);
    else params.log.info(`[plugins] ${diag.message}`);
  }

  return { pluginRegistry, gatewayMethods };
}
```

本节小结

1. **OpenClaw 的通道分为核心通道和扩展通道**，两者能力等价，区别在于部署策略。扩展通道通过插件系统按需安装，减小主包体积并支持独立版本管理。
2. **插件加载流程遵循"发现→解析→加载→注册"四阶段**：从四个位置发现候选插件，读取 `openclaw.plugin.json` 清单，通过 jiti 动态加载 TypeScript/JavaScript 模块，最后调用 `register()` 函数完成注册。
3. **jiti 运行时允许直接加载 TypeScript**，配合 `openclaw/plugin-sdk` 别名映射，使得插件开发无需配置构建工具链。
4. **`OpenClawPluginApi` 是插件与主程序的唯一交互接口**，采用控制反转（IoC）模式，插件通过注册回调声明能力，由运行时在合适时机调用。
5. **插件清单是必需的**，它定义了插件 ID、配置 Schema、通道/Provider 声明等元信息，是插件系统进行验证和管理的基础。